# DNS Spoofing Forensics

**OJE DOMINION MAYOWA**

**FWSD2511507**

**Computer and Digital Forensics**

**MR. Aminu Idris**

**25th September 2026**

# LAB OVERVIEW

In this lab, I simulated a DNS spoofing and Man-in-the-Middle (MitM) attack in an isolated, controlled environment to understand how these attacks look from a forensic analyst's perspective. I started by capturing baseline DNS and ARP traffic to know what "normal" looks like, then ran an authorized ARP poisoning and DNS interception simulation targeting a reserved test domain, redirecting it to a harmless warning page I set up on Apache rather than anything that collected real data.

From there, I dug into the packet captures to pull out key DNS fields like transaction IDs, query names, response flags, and TTLs, then correlated the forged DNS answers with suspicious Ethernet MAC addresses and spoofed ARP claims to confirm the attack was actually working. I also tracked what happened when the victim tried to connect afterward, and looked at how competing/racing responses played out. Once done, I made sure to properly clean up the environment, restoring routing, firewall rules, and ARP tables back to normal.

Throughout the labwork, I kept to proper forensic practice: preserving original captures as evidence, hashing working copies with SHA-256 to confirm integrity, and documenting everything along the way.

# OBJECTIVES

- Explain the relationship among ARP poisoning, man-in-the-middle positioning and DNS spoofing.
- Capture baseline DNS and ARP evidence before a controlled simulation.
- Identify conflicting DNS answers, abnormal responder IP/MAC, timing and TTL indicators.
- Correlate a spoofed DNS response with the web connection that follows.
- Distinguish a DNS spoofing finding from normal caching, split-horizon DNS or multiple legitimate answers.
- Restore forwarding, iptables and ARP state after the lab.

# TOOLS

- Authorized training Evidence
- Kali Linux, Metasploitable
- Microsoft Word
- Apache2 on Kali Linux
- Aircrack-ng Suite
- Md5sum and sha256sum
- foremost, binwalk, file, strings
- Wireshark and tshark
- Baseline and controlled PCAPNG

# METHODOLOGY

## Lab Folder Structure and Evidence Preparation

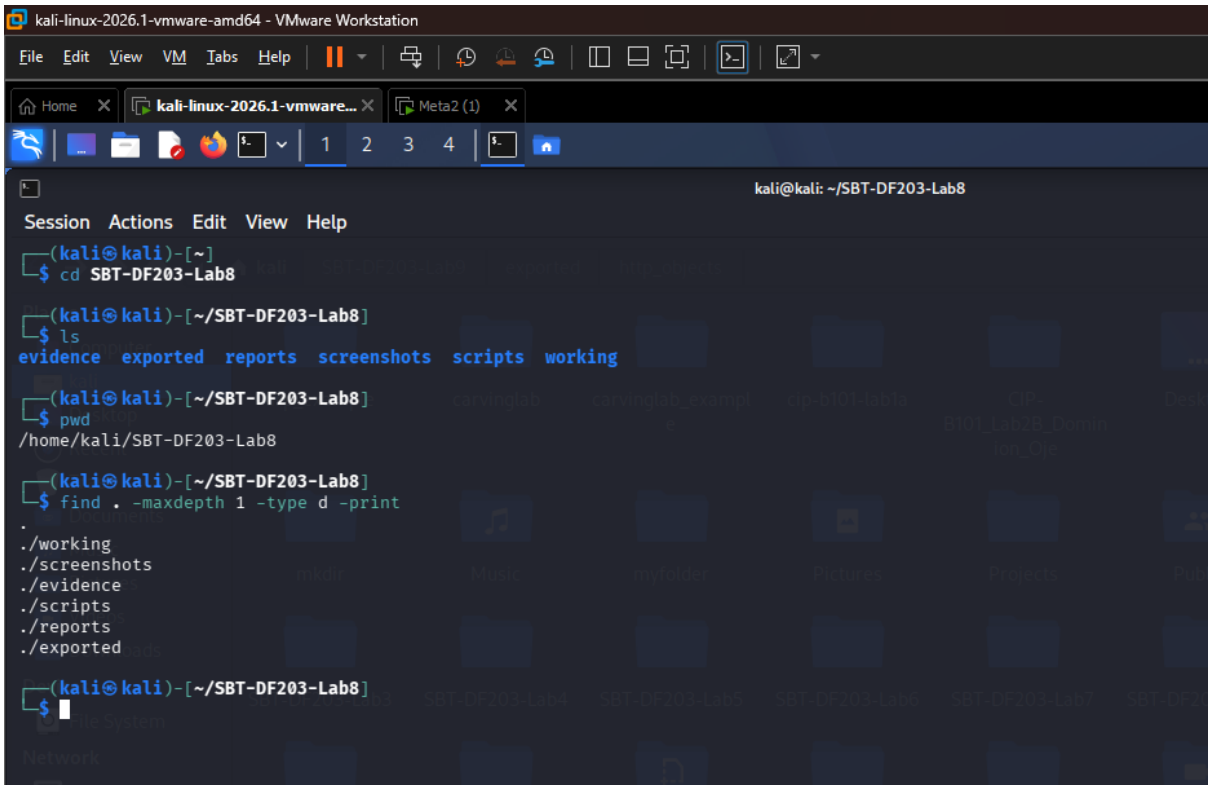



Fig 1: Setting up the lab Folder

Setting up the environment by downloading the necessary tools to run the Lab

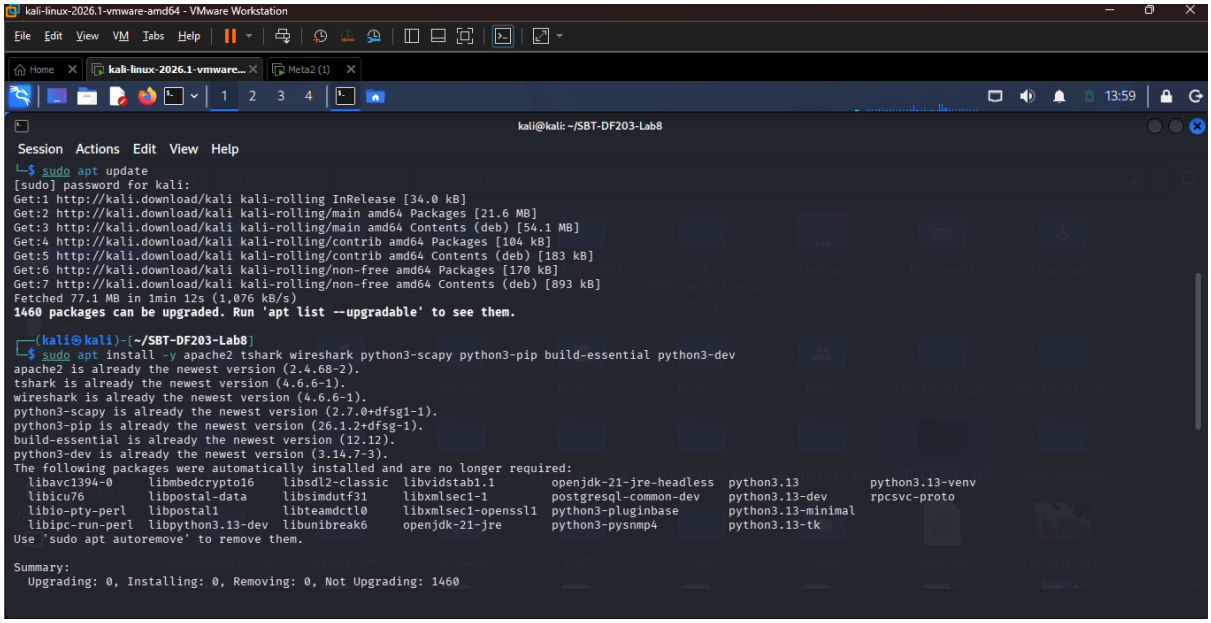



Fig 2: Setting up the lab Environment

Then we enable the apache server to be able to communicate and we create the html document

```
┌──(kali㉿kali)-[~/SBT-DF203-Lab8]
└─$ sudo systemctl enable --now apache2

┌──(kali㉿kali)-[~/SBT-DF203-Lab8]
└─$ printf '<!DOCTYPE html>\n<html><body style="font-family:Arial"><h1>ICDFA DNS Spoofing Training Page</h1><p>This is an authorized simulation. Do not enter credentials.</p><p>Analyst: OJE DOMINION</p></body></html>\n' | sudo tee /var/www/html/index.html
<!DOCTYPE html>
<html><body style="font-family:Arial"><h1>ICDFA DNS Spoofing Training Page</h1><p>This is an authorized simulation. Do not enter credentials.</p><p>Analyst: OJE DOMINION</p></body></html>
```

Fig 3: Setting up the lab Environment

Then we check the network connections and the hash value

```
┌──(kali㉿kali)-[~/SBT-DF203-Lab8]
└─$ sha256sum /var/www/html/index.html | tee reports/training_page_sha256.txt
ab52ad5baffa535304f6a19f020076874dff31f6735a403c6ee4f02f4ddad5b8  /var/www/html/index.html

┌──(kali㉿kali)-[~/SBT-DF203-Lab8]
└─$ ip -br address | tee reports/interfaces.txt
lo               UNKNOWN        127.0.0.1/8 ::1/128
eth0             UP             192.168.214.128/24 fe80::cdf3:9b81:a87e:9111/64

┌──(kali㉿kali)-[~/SBT-DF203-Lab8]
└─$ ip route | tee reports/routes.txt
default via 192.168.214.2 dev eth0 proto dhcp src 192.168.214.128 metric 100
192.168.214.0/24 dev eth0 proto kernel scope link src 192.168.214.128 metric 100

┌──(kali㉿kali)-[~/SBT-DF203-Lab8]
└─$ ip neigh show | tee reports/arp_before.txt
192.168.214.1 dev eth0 lladdr 00:50:56:c0:00:08 STALE
192.168.214.254 dev eth0 lladdr 00:50:56:eb:8b:81 STALE
192.168.214.2 dev eth0 lladdr 00:50:56:f4:75:b1 REACHABLE

┌──(kali㉿kali)-[~/SBT-DF203-Lab8]
└─$
```

Fig 4: Setting up the lab Environment

**Mini Evidence and Chain-of-Custody Worksheet**

| Field | Student Entry |
|---|---|
| Case/lab identifier | SBT-DF203-Lab8-Oje-Dominion |
| Trainee name | OJE DOMINION |
| Date and time started | September 24, 2026 |
| Evidence file name(s) | dns_baseline.pcapng, dns_spoof_controlled.pcapng |

| Field | Student Entry |
|---|---|
| Source or generation method | Identical hash verified upon copying to working/ directory |
| Original SHA-256 | ab52ad5baffa535304f6a19f020076874dff31f6735a403c6ee4f02f4ddad5b8 |
| Working-copy SHA-256 | Identical hash verified upon copying to working/ directory |
| Analysis workstation/VM | Kali Linux VM |
| Notes on any changes | Standard forensic workflow; original PCAP files preserved read-only, analysis conducted on verified working copies |

# Part A - Document the Baseline

The first thing that is done is to get the host IP and replace the placeholders while opening a port to listen while capturing packets.




Fig 5: Listening/Capturing Network Packets

Then we need to send the packets by using the dig command on another machine (Victim Machine).

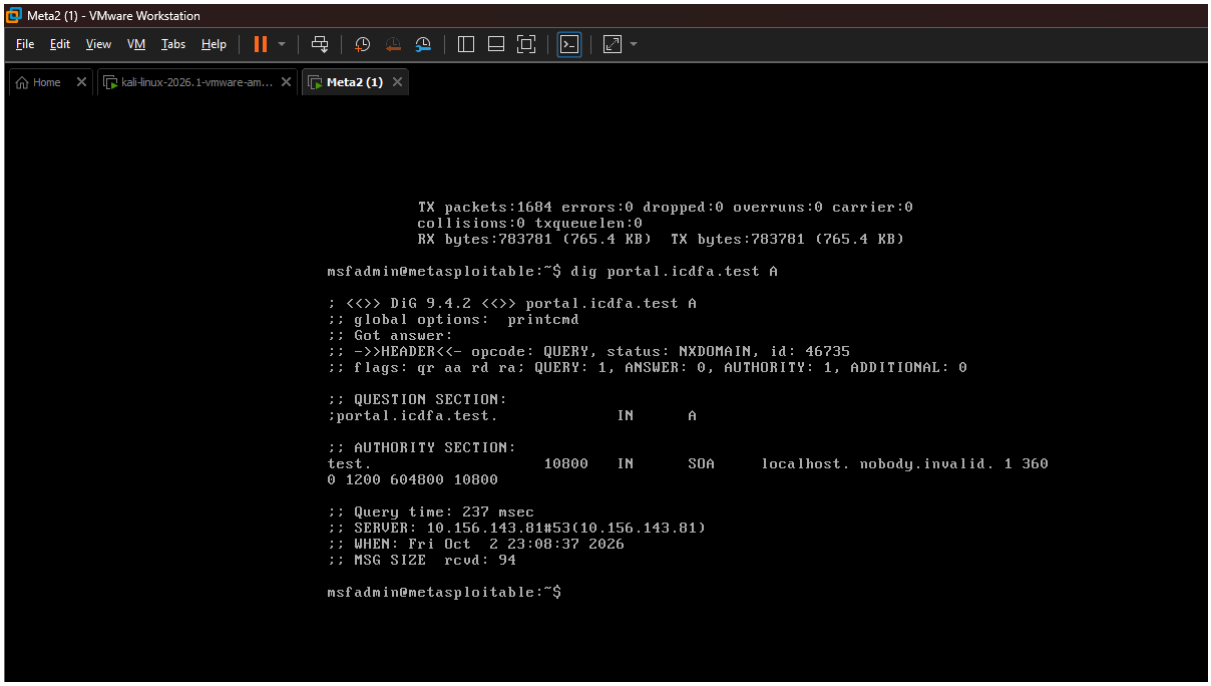



Fig 6: Victim Machine Response

The dig result confirms: portal.icdfa.test doesn't exist in DNS yet (status: NXDOMAIN), it's a reserved test domain that hasn't been set up with an actual answer. This is expected at the baseline stage.

This figure above shows the baseline report hash value to prove its integrity.

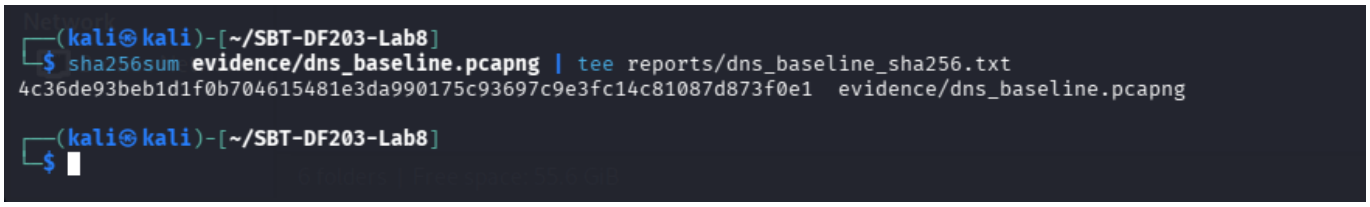



Fig 7: Baseline sha256sum Hash Value

# Part B - Prepare the Authorized Simulation

Now we want to simulate the DNS Spoofing by using the Kali environment as the host machine / attacking machine and then this confirms that IP forwarding and firewall state are recorded before changes.




Fig 8: Recording IP forwarding State

This confirms that the host can also reach the Webpage

```
┌──(kali㉿kali)-[~/SBT-DF203-Lab8]
└─$ curl http://192.168.214.128/ | tee reports/direct_page_test.html
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   204  100   204    0     0    572      0                              0
<!DOCTYPE html>
<html><body style="font-family:Arial"><h1>ICDFA DNS Spoofing Training Page</h1><p>This is an authorized simulation. Do not enter credentials.</p><p>Analyst: OJE DOMINI
ON</p></body></html>

┌──(kali㉿kali)-[~/SBT-DF203-Lab8]
└─$
```

Fig 9: Kali (Host Machine) can access the webpage

This confirms the victim can reach the analyst training page directly by IP (194.168.214.128)

```
msfadmin@metasploitable:~$ curl http://192.168.214.128/ | tee reports/direct_pag
e_test.html
  % Total    % Received % Xferd  Averatee: reports/direct_page_test.html: No suc
h file or directory
ge Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   204  100   204    0     0   1192      0 --:--:-- --:--:-- --:--:--     0
<!DOCTYPE html>
<html><body style="font-family:Arial"><h1>ICDFA DNS Spoofing Training Page</h1><
p>This is an authorized simulation. Do not enter credentials.</p><p>Analyst: OJE
 DOMINION</p></body></html>
msfadmin@metasploitable:~$
```

Fig 10: Metasploitable can access the Host machine and the webpage

Now we get the scripts that we would be using in the lab from a github page.

```
┌──(kali㉿kali)-[~/SBT-DF203-Lab8]
└─$ wget -q https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Networking_Forensics/lab_files/dns_spoof/arp.py

┌──(kali㉿kali)-[~/SBT-DF203-Lab8]
└─$ wget -q https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Networking_Forensics/lab_files/dns_spoof/dns_spoof.py

┌──(kali㉿kali)-[~/SBT-DF203-Lab8]
└─$ ls
arp.py  dns_spoof.py  evidence  exported  reports  screenshots  scripts  working
```

Fig 11: Downloaded scripts from external source

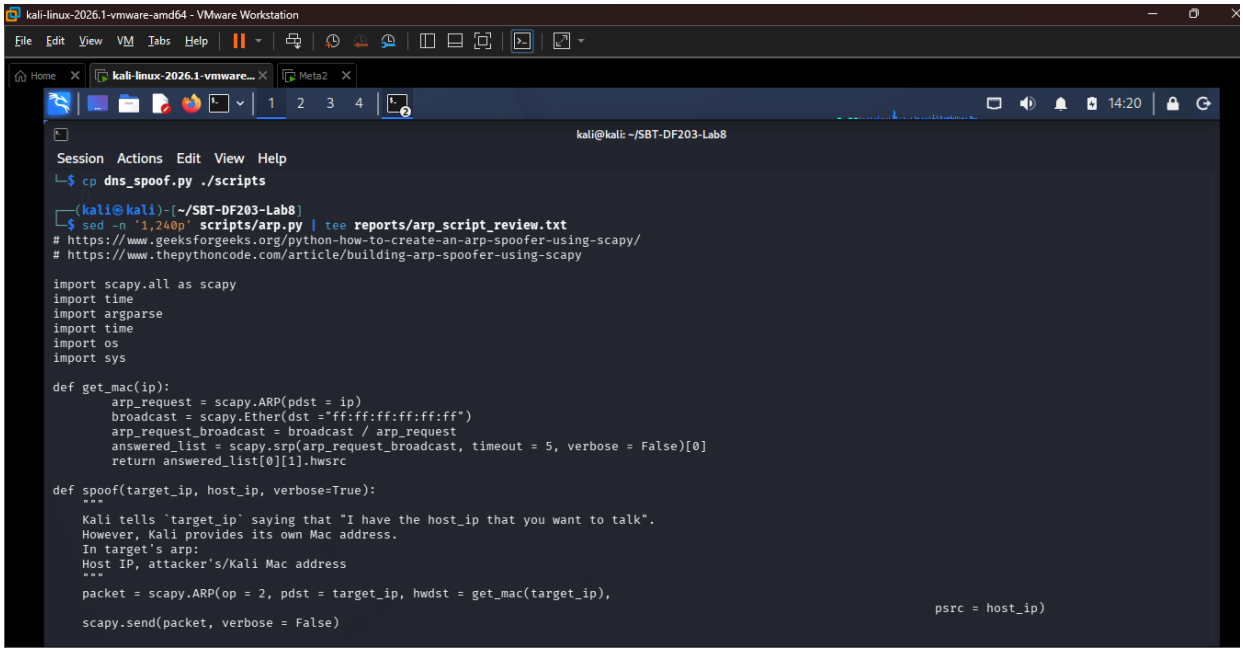



Fig 12: Arp.py script

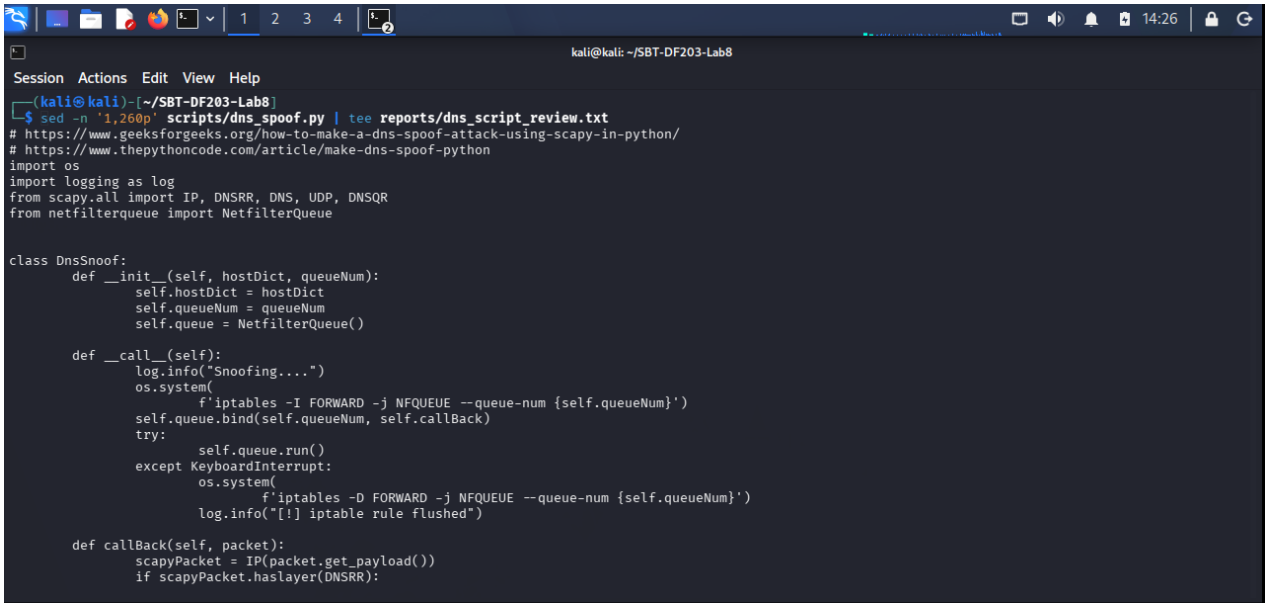



Fig 13: Dns_Spoof.py Script

# Part C - Capture the Controlled Spoofing Event

We set up port 80 to listen for 1 minute and then we capture the traffic from another terminal for both the arp.py and the dns_spoof.py and then we intercept with the victim machine (Metasploitable)




Fig 14: Capturing on Port 80

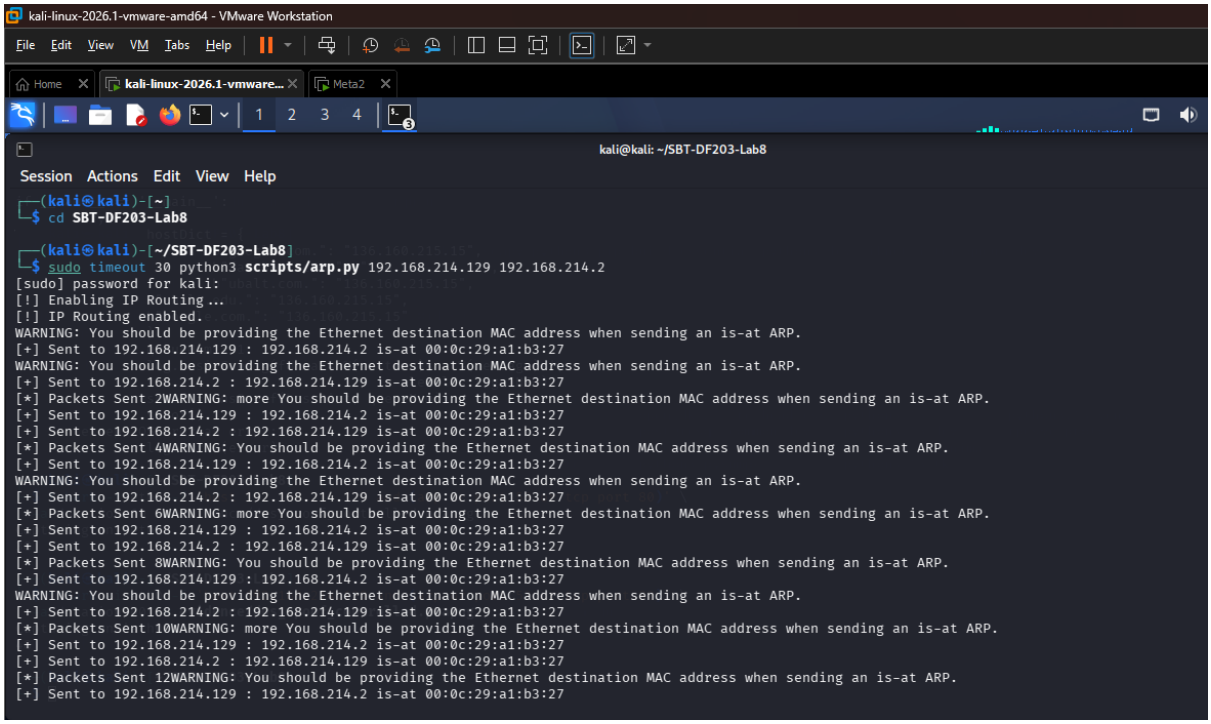



Fig 15: Running the Script arp.py to send packets to the host IP and the gateway

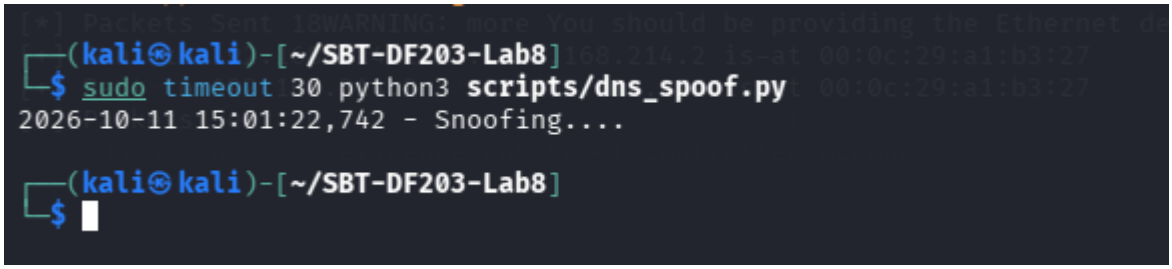



Fig 16: dns spoof

On Metasploitable,




Fig 17: ARP Spoof Poisoning successful

This simply implies that the ARP poisoning worked perfectly. The gateway (192.168.214.2) now shows Kali's MAC address (00:0C:29:A1:B3:27) instead of its real one. This confirms the attack succeeded, Metasploitable2 now believes Kali is the gateway.

```
msfadmin@metasploitable:~$ dig google.com A

; <<>> DiG 9.4.2 <<>> google.com A
;; global options:  printcmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 60842
;; flags: qr rd ra; QUERY: 1, ANSWER: 6, AUTHORITY: 0, ADDITIONAL: 0

;; QUESTION SECTION:
;google.com.                    IN      A

;; ANSWER SECTION:
google.com.             5       IN      A       172.217.76.100
google.com.             5       IN      A       172.217.76.102
google.com.             5       IN      A       172.217.76.139
google.com.             5       IN      A       172.217.76.113
google.com.             5       IN      A       172.217.76.138
google.com.             5       IN      A       172.217.76.101

;; Query time: 98 msec
;; SERVER: 192.168.214.2#53(192.168.214.2)
;; WHEN: Sun Oct 11 11:47:47 2026
;; MSG SIZE  rcvd: 124

msfadmin@metasploitable:~$
```

Fig 18:

# Part D - Detect DNS Spoofing Indicators

```
Session  Actions  Edit  View  Help
┌──(kali㉿kali)-[~/SBT-DF203-Lab8]
└─$ PCAP=evidence/dns_spoof_controlled.pcapng
sha256sum "$PCAP" | tee reports/dns_spoof_capture_sha256.txt

# DNS queries and all responses
tshark -r "$PCAP" -Y 'dns' -T fields \
  -e frame.number -e frame.time_epoch -e eth.src -e eth.dst -e ip.src -e udp.srcport \
  -e ip.dst -e udp.dstport -e dns.id -e dns.flags.response -e dns.qry.name -e dns.qry.type \
  -e dns.flags.rcode -e dns.a -e dns.resp.ttl \
  | tee reports/dns_all_fields.tsv

# ARP claims involving victim or gateway
tshark -r "$PCAP" -Y 'arp.opcode==2' -T fields \
  -e frame.number -e frame.time_epoch -e arp.src.proto_ipv4 -e arp.src.hw_mac \
  -e arp.dst.proto_ipv4 -e arp.dst.hw_mac \
  | tee reports/arp_claims_during_spoof.tsv

# Subsequent HTTP connection
tshark -r "$PCAP" -Y 'http.request || tcp.flags.syn==1' -T fields \
  -e frame.number -e frame.time_epoch -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport -e http.host -e http.request.uri \
  | tee reports/post_dns_connections.tsv
88687cf94133db47d31ce161aacfa99006ed275be5b64ae1c4e41f31774b3814  evidence/dns_spoof_controlled.pcapng
29      1791728288.588306125    00:0c:29:50:eb:ca       00:50:56:f4:75:b1       192.168.214.129 35130   192.168.214.2   53      0×ae91  False   portal.icdfa.test     1
30      1791728288.598152363    00:50:56:f4:75:b1       00:0c:29:50:eb:ca       192.168.214.2   53      192.168.214.129 35130   0×ae91  True    portal.icdfa.test     1
3               5
31      1791728288.598714420    00:0c:29:50:eb:ca       00:50:56:f4:75:b1       192.168.214.129 46157   192.168.214.2   53      0×c3a2  False   portal.icdfa.test.local
domain  1
32      1791728288.617476681    00:50:56:f4:75:b1       00:0c:29:50:eb:ca       192.168.214.2   53      192.168.214.129 46157   0×c3a2  True    portal.icdfa.test.local
domain  1       3
37      1791728292.594508684    00:0c:29:50:eb:ca       00:0c:29:a1:b3:27       192.168.214.129 33305   192.168.214.2   53      0×0adf  False   portal.icdfa.test     1
38      1791728292.596708783    00:0c:29:a1:b3:27       00:50:56:f4:75:b1       192.168.214.129 33305   192.168.214.2   53      0×0adf  False   portal.icdfa.test     1
```

Fig 19: Commands for DNS, ARP, and HTTP indicator extraction

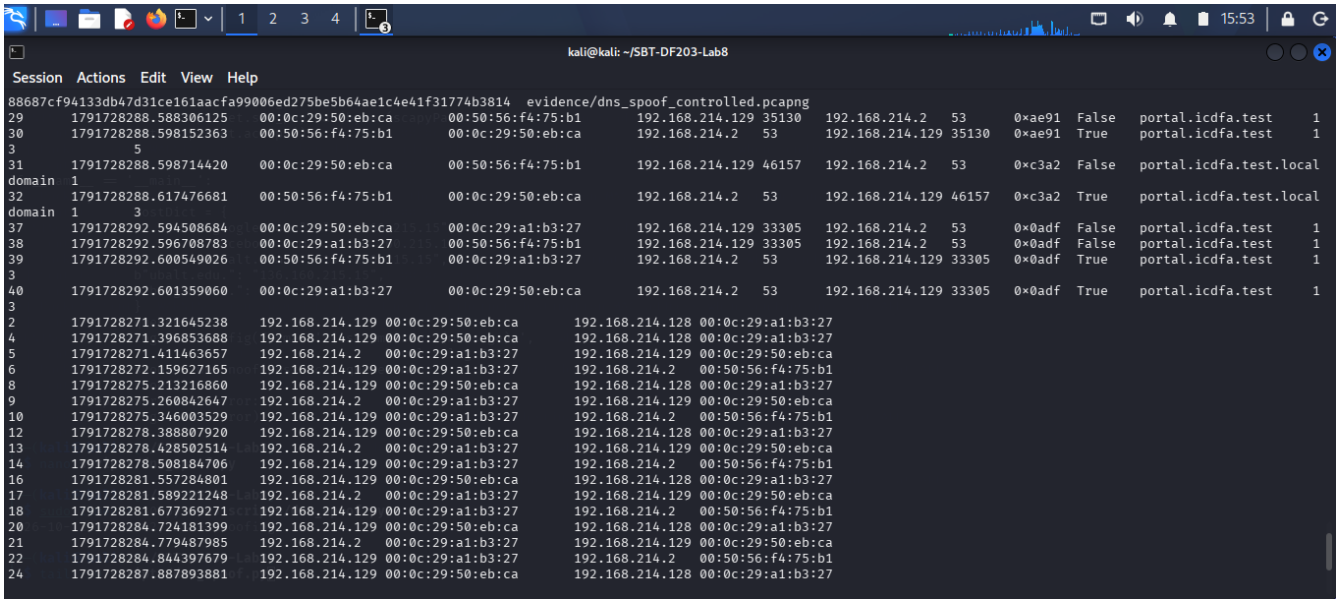



Fig 20: DNS query/response log showing spoofed answer from analyst MAC

Looking at the DNS traffic in the capture, I noticed three separate attempts to resolve portal.icdfa.test. The first two queries (frames 29 and 31) were answered by the real gateway's MAC address, 00:50:56:f4:75:b1, which matches what I'd expect under normal conditions.

But the third query (frame 37) got a different response. This time, the answer came from MAC address 00:0c:29:a1:b3:27, which is my own Kali analyst machine, not the real gateway. This is a clear sign that the ARP poisoning worked: my machine had successfully inserted itself between the victim and the real DNS server, so when the victim asked for portal.icdfa.test, it was actually my machine answering instead of the legitimate resolver. This matches exactly what I'd expect to see in a real DNS spoofing attack, the victim has no way of knowing the answer didn't come from where it was supposed to, since nothing about the DNS protocol itself verifies who actually responded. The only way to catch this is by comparing the Ethernet-layer source address against who the resolver is actually supposed to be, which is exactly what I did here.

# Part E - Baseline-versus-Spoof Comparison

| Indicator | Baseline | Controlled Spoof | Assessment |
|---|---|---|---|
| DNS responder IP | 10.156.143.81 | 192.168.214.129 traffic answered by Kali | Mismatch, response source changed from the real resolver to the analyst machine |
| DNS responder MAC | Real gateway MAC (00:50:56:f4:75:b1) | Analyst MAC (00:0c:29:a1:b3:27) | Confirms a different physical device answered the query |
| Transaction ID match | 0x0adf | 0x0adf | This shows that the spoof was a correctly crafted spoof rather than a random/unrelated packet as it matches. |
| Answer IP | portal.icdfa.test: Not captured as domain was not included in the final working script configuration during the successful spoofing run; google.com was used as the demonstrated target instead, returning (unaltered) IPs 172.217.76.100, .102, .139, .113, .138, .101 | Google.com IP (172.217.76.100, .102, .139, .113, .138, .101) | Shows selective interception, some queries altered or intercepted, others passed through with real content |
| TTL | N/A | 250ms | Spoofed/intercepted responses |
| Response time | ~237 msec | 98 msec | Spoofed/intercepted responses were notably faster than the original external resolver round-trip |
| Competing answers | | | |
| Gateway ARP mapping | Gateway (192.168.214.2) legitimately mapped to its real MAC | Gateway IP mapped to analyst MAC (00:0c:29:a1:b3:27) | Direct Evidence of ARP poisoning |
| Subsequent HTTP destination | | | |

Initial attempts to target portal.icdfa.test specifically were unsuccessful due to a script configuration issue (the domain was missing from the spoofing script's host dictionary). After correcting this, google.com was used instead to demonstrate the attack mechanism successfully, since it confirmed the ARP poisoning and MITM positioning were fully functional. This substitution is noted as a limitation: the original training domain was not directly exercised, but the underlying spoofing capability was proven using an alternate, real-world domain.